



## 1 Introdução

Este documento descreve a implementação de um simulador de alocação dinâmica de memória contígua com partições variáveis. O programa lê um arquivo de entrada com o tamanho da memória e uma sequência de eventos (alocações e liberações de processos), executa a simulação passo a passo e exibe o estado da memória após cada operação.

O simulador implementa três estratégias de alocação: First Fit, Best Fit e Worst Fit. As definições seguem [1], capítulo 9, seções 9.2.2 e 9.2.3.

O código foi escrito em C (padrão C99), compilado com GCC e organizado em módulos com um header único. A interface é textual, executada em terminal.

## 2 Repositório

O código-fonte completo está disponível no repositório GitHub:

<https://github.com/luizfpq/ufms-facom-so-2026.1-t2>

O repositório permaneceu **privado** durante toda a execução do projeto, sendo tornado público a partir das 23:59 do dia da entrega, permanecendo acessível para auditoria até que o docente lance as notas ou solicite a retirada. Essa decisão visa facilitar a rastreabilidade do código comitado, permitindo a verificação do histórico de desenvolvimento via `git log`. Caso o docente prefira, o acesso pode ser revogado após a avaliação.

## 3 Algoritmos de Alocação

Os três algoritmos compartilham a mesma estrutura de dados (vetor de blocos contíguos) e diferem apenas no critério de seleção do bloco livre.

**First Fit.** Percorre a lista de blocos do início ao fim. Aloca no primeiro bloco livre com tamanho suficiente. Complexidade  $O(n)$  por operação. Tende a fragmentar o início da memória, porque alocações consecutivas se concentram nas primeiras brechas disponíveis.

**Best Fit.** Percorre toda a lista e escolhe o menor bloco livre que comporte o processo. Complexidade  $O(n)$ . Minimiza o desperdício imediato, mas gera brechas pequenas que dificilmente serão reutilizadas. Segundo [1], pode produzir fragmentação pior que o First Fit em cargas de trabalho com alocações de tamanhos variados.

**Worst Fit.** Percorre toda a lista e escolhe o maior bloco livre disponível. Complexidade  $O(n)$ . A ideia é que a brecha restante seja grande o suficiente para atender outros processos. Na prática, consome rapidamente os blocos grandes e pode forçar falhas de alocação mais cedo.

## 4 Estrutura do Projeto

A organização segue o princípio de um arquivo por responsabilidade.

```
1 memoria.h          - Structs e prototipos (header unico)
2 main.c             - Ponto de entrada e dispatch
3 arquivo.c          - Parser do arquivo de entrada
4 memoria_ops.c      - Algoritmos de alocação e liberação
5 visualizacao.c     - Interface textual e relatorio
6 Makefile           - Compilação e execução
7 entrada_*.txt      - Cenários de teste (first, best, worst, comparativo, robustez)
8 resultados/        - Saídas reproduzíveis de cada algoritmo
```

A memória é representada por um vetor estático de blocos. Cada bloco possui endereço inicial, tamanho e identificador do processo ocupante (zero indica bloco livre). Quando um processo é alocado em um bloco maior que o necessário, o bloco é dividido em dois: a parte ocupada e o restante livre. Quando um processo é liberado, blocos livres adjacentes são mesclados automaticamente.

```
1 typedef struct {
2     int inicio;          /* endereco inicial */
3     int tamanho;
4     int pid;             /* id do processo, 0 = livre */
5 } Bloco;
```

O dispatch de algoritmos usa function pointers. O main atribui a função de alocação uma vez (conforme o cabeçalho do arquivo de entrada) e a reutiliza em todos os eventos.

## 5 Compilação e Uso

### 5.1 Pré-requisitos

- GCC (GNU Compiler Collection)
- GNU Make

### 5.2 Compilação

Ao compilar, o Makefile exibe automaticamente as opções de uso disponíveis:

```
1 make
```

### 5.3 Execução

```
1 ./simulador <arquivo_de_entrada.txt>
2 ./simulador <arquivo_de_entrada.txt> -a ALGORITMO
```

A flag `-a` permite sobrescrever o algoritmo definido no cabeçalho do arquivo de entrada, possibilitando executar qualquer cenário com qualquer estratégia sem editar o arquivo:

```
1 ./simulador entrada_comparativo.txt -a WORST_FIT
```

Atalhos via Make:

```
1 make first          # First Fit
2 make best           # Best Fit
3 make worst          # Worst Fit
4 make comparativo    # Tres algoritmos na mesma entrada
5 make help           # Exibe opcoes disponiveis
```

O formato do arquivo de entrada é simples. Linhas com `#` configuram parâmetros ou são ignoradas. Cada evento é uma linha com comando, ID do processo e tamanho (quando aplicável):

```

1 # Algoritmo: FIRST_FIT
2 # Memoria: 1000
3 ALOCAR P1 200
4 ALOCAR P2 300
5 LIBERAR P1
6 ALOCAR P3 250

```

Algoritmos aceitos: FIRST\_FIT, BEST\_FIT, WORST\_FIT.

## 6 Visualização

O simulador exibe após cada passo: (1) tabela com todos os blocos, indicando se estão ocupados ou livres, com endereço e tamanho; (2) mapa visual compacto (caracteres . para livre, dígito do ID para ocupado); (3) resultado da operação (sucesso ou falha, com indicação de fragmentação externa quando aplicável).

Exemplo de saída após três alocações e uma liberação:

```

1 [5] LIBERAR P2
2
3 [Passo 5] Memoria (1000 unidades):
4 +-----+
5 | P1      inicio=0      tam=200      |
6 | LIVRE   inicio=200    tam=300      |
7 | P3      inicio=500    tam=150      |
8 | LIVRE   inicio=650    tam=350      |
9 +-----+
10 1111111111.....3333333.....

```

O relatório final apresenta: memória total, espaço ocupado e livre, número de brechas, percentual de utilização, total de alocações que falharam e fragmentações externas detectadas.

A fragmentação externa é detectada quando a soma dos blocos livres é suficiente para atender a requisição, mas nenhum bloco individual comporta o processo. O simulador reporta essa condição com mensagem própria, distinta da falha por memória insuficiente. Cada alocação tem um de cinco desfechos: sucesso, falha por fragmentação externa, falha por memória insuficiente, rejeição por tamanho inválido ou rejeição por processo já alocado.

## 7 Análise Comparativa

Para evidenciar as diferenças entre os algoritmos, os três foram executados com a mesma entrada: memória de 1000 unidades e 13 eventos projetados para provocar fragmentação (alocações de tamanhos variados intercaladas com liberações não sequenciais).

Tabela 1: Resultados comparativos com o cenário de teste.

| Métrica                | First Fit | Best Fit | Worst Fit |
|------------------------|-----------|----------|-----------|
| Utilização final       | 43,0%     | 43,0%    | 43,0%     |
| Brechas ao final       | 3         | 3        | 3         |
| Alocações que falharam | 2         | 2        | 2         |
| Fragmentações externas | 1         | 1        | 1         |

No cenário testado, os três algoritmos apresentaram os mesmos totais de falhas. A diferença está no posicionamento dos blocos ao longo da simulação.

First Fit e Best Fit alocaram P5 (250 unidades) na brecha de 300 deixada por P2 (posição 200). A brecha de 300 era simultaneamente a primeira e a menor que comportava 250, por isso os dois algoritmos coincidiram.

Worst Fit alocou P5 na brecha final (maior contíguo), posicionando-o mais adiante na memória. Isso preservou o espaço no início, mas consumiu a maior brecha disponível.

A fragmentação externa ocorreu no passo 13 (P9, 400 unidades): havia 570 unidades livres distribuídas em 3 brechas, nenhuma com 400 contíguos. Esse resultado é idêntico nos três algoritmos porque a sequência de liberações criou o mesmo padrão de brechas residuais.

As saídas completas de cada execução estão disponíveis na pasta **resultados/** do repositório, permitindo reprodução e verificação independente via **make comparativo**.

## 8 Decisões de Implementação

- **Vetor estático de blocos.** Limite de 200 blocos e 100 eventos. Suficiente para simulação didática e evita complexidade de alocação dinâmica (**malloc/free**) no próprio simulador de memória.
- **Mesclagem automática.** Após cada liberação, blocos livres adjacentes são fundidos. Reduz fragmentação sem intervenção manual e mantém a lista compacta.
- **Divisão de bloco.** Quando sobra espaço após alocação, um novo bloco livre é inserido na posição seguinte. A lista permanece sempre ordenada por endereço.
- **Deteção de fragmentação.** Soma todos os blocos livres e compara com a requisição. Se o total é suficiente mas nenhum individual comporta, reporta fragmentação externa.

## 9 Revisão Crítica e Correções

Antes de fechar o trabalho, revisei o simulador contra o enunciado, evento por evento. A primeira versão passava nos casos normais. Falhava nos casos de borda. Registro aqui o que encontrei e como corrigi.

**Mensagem de falha ambígua.** O enunciado pede para sinalizar fragmentação externa: quando há espaço total, mas não contíguo. A primeira versão rotulava toda falha como “sem bloco contíguo”. Isso mistura duas causas distintas. Pedir 2000 unidades em uma memória de 1000 não é fragmentação. É falta de memória. Separei os dois casos. Agora o simulador informa “fragmentacao externa” quando o espaço total existe, e “memoria livre insuficiente” quando não existe.

**Tamanho inválido.** A primeira versão aceitava **ALOCAR** com tamanho zero. Criava uma partição vazia. Adicionei validação: tamanho menor ou igual a zero é rejeitado antes de tentar alocar.

**Processo duplicado.** Dois **ALOCAR** com o mesmo ID eram aceitos. O **LIBERAR** seguinte só desalocava o primeiro bloco. Adicionei checagem de unicidade. Um ID já presente na memória é rejeitado.

**Truncamento silencioso.** O parser ignorava eventos acima do limite de 100 sem avisar. Agora emite alerta quando atinge o limite.

Criei o cenário **entrada\_robustez.txt** para exercitar todos esses casos de uma vez. A saída está em **resultados/robustez.txt**. Roda com **make robustez**.

A lição foi clara. O difícil não foi implementar os algoritmos. A lógica de First, Best e Worst Fit segue o livro. O difícil foi tratar as entradas que o usuário não deveria mandar, mas manda. A robustez veio dos testes de borda, não da implementação inicial.

## 10 Limitações

- Não implementa compactação de memória, paginação ou segmentação.
- Cada processo ocupa exatamente um bloco contíguo.

- Não há modelagem de tempo de acesso nem overhead de gerenciamento.

Essas simplificações são coerentes com o escopo do enunciado, que trata exclusivamente de alocação contígua em esquema de partições variáveis.

## 11 Declaração de Uso de Inteligência Artificial

Em conformidade com a Resolução da UFMS publicada no Boletim Oficial (ID 581705), que regula-menta o uso de ferramentas de inteligência artificial em atividades acadêmicas, declaro que:

- Os **dados de entrada** (arquivos `entrada_*.txt`) foram formatados e sugeridos com auxílio da ferramenta **Kiro CLI** (Anthropic), utilizada como apoio na geração de cenários de teste que evidenciem fragmentação externa e diferenças entre os algoritmos.
- Os **formatos dos dados de saída** foram refatorados no código C com auxílio da ferramenta **Kiro CLI** (Anthropic), para facilitar a visualização do estado da memória a cada passo.
- A **revisão crítica** do código contra o enunciado e a implementação das **correções de robustez** (mensagens de falha distintas, validação de tamanho, processo duplicado e truncamento) contaram com apoio da ferramenta **Kiro CLI** (Anthropic), que atuou como revisora e apontou os casos de borda. A decisão sobre o que corrigir e como tratar cada caso foi minha.
- A **lógica dos algoritmos**, a **estrutura do código** e a **análise dos resultados** são de autoria própria, baseadas nas referências bibliográficas citadas neste documento.

## Referências

- [1] Abraham Silberschatz, Peter Baer Galvin e Greg Gagne. *Operating System Concepts*. 10<sup>a</sup> ed. Hoboken: John Wiley & Sons, 2018.